

Daily Progress & Next Steps Plan

Constraint-Interaction Metric for Symbolic Regression

Biswajyoti Nath

Subinoy Nath

July 9, 2026

Part 1: Summary of Today's Work (Monte Carlo Estimator)

Today's focus was on implementing the mathematical core of the framework: the Monte Carlo density estimator and the interaction matrix module (`src/metric.py`).

- **Density Estimator:** Implemented Algorithm 1 from the Research Plan. The `DensityEstimator` correctly evaluates $\rho(C_i)$ and $\rho(C_i \wedge C_j)$ in a single $O(N \cdot k^2)$ pass over N valid stratified expressions.
- **Matrix Symmetry & Edge Cases:** Built the `InteractionMatrix` to explicitly mirror the upper triangle to the lower triangle, strictly enforcing the $M[i][j] = M[j][i]$ invariant. Handled zero-density components gracefully by propagating `NaN` rather than causing divide-by-zero crashes.
- **Statistical Rigor:** Implemented `BootstrapCI` using the percentile method ($B = 1000$ resamples) to attach 95% confidence intervals to the interaction coefficients.
- **Calibration-First Benchmarking:** Created a `run_benchmark()` utility that logs hardware-specific generation metrics (wall-clock time, rejection rates) to `results/benchmark_1k.json`. This establishes an empirical baseline before scaling up to $N = 100,000$.
- **Regression Testing:** Added a 15-test suite for `metric.py` (bringing the project total to 33 passing tests). Critically, included a `sympify(evaluate=False)` structural regression test to guarantee that abstract syntax trees (like `x + x`) do not collapse into simplified algebraic forms (`2*x`) before constraint evaluation.

Part 2: Strict Plan for Tomorrow

With the `expr_generator` and `metric.py` modules complete and rigorously tested, tomorrow's efforts will focus on implementing the **Constraint Evaluators** and executing the first **Full Interaction Analysis**.

Phase 1: Implement Constraints (`src/constraints.py`)

We will build the pure, deterministic Python functions that categorize the expressions.

- C1. **Structural Constraints:** Ensuring specific variable presence or polynomial shapes.
- C2. **Depth Constraints:** Filtering ASTs that exceed strict complexity limits.
- C3. **Operator Whitelists:** Validating that only subset operators (e.g., `{+, *, sin}`) exist in the tree.

- C4. **Positivity Constraints:** Ensuring the domain of the expression is strictly non-negative.
- C5. **Dimensional Constraints:** Enforcing specific physical dimensional consistency on the AST leaves.

Phase 2: The 100k Monte Carlo Run

Integrate the fully specified constraints with the Monte Carlo pipeline.

- Execute the density estimator at scale ($N = 100,000$).
- Generate the first complete Interaction Matrix heatmap $M(i, j)$.
- Package all generator stats, metadata, and RNG seeds into the Reproducibility Contract JSON logging payload.

Phase 3: PySR Integration (`src/experiments.py`)

Begin scaffolding the interface that binds these Python-defined constraints into PySR's Julia-backed evolutionary search engine.